

Android Forensic Analysis Using Open-Source Tools

Progress Report

Umesh Kalupathirannehelage – 300389749

Group 7 (F25_4440_G7)

Mobile Cybersecurity

CSIS4440 - 003

1. Work Log Table

Student Name: Umesh Kalupathirannehelage

Date	Student Name	Description of Work Done	Hours
6th Oct 2025	Umesh	Researched tools and apps suitable for the project.	1.0
7th Oct 2025	Umesh	Drafted an email to professor to finalize tools and applications.	0.5
9th Oct 2025	Both	Met with professor to discuss project scope and confirm selected tools & apps.	0.5
21st Oct 2025	Umesh	Researched and set up ALEAPP and MobSF; drafted extraction workflow for Dropbox & Strava.	1.0
28th Oct 2025	Umesh	Installed Android SDK command-line tools and configured AVD, ADB, SDK Manager, and APK Analyzer paths. Created and launched emulator. Installed Dropbox and Strava APKs for testing.	2
29th Oct 2025 (Night)	Umesh	Created an email account, separate Strava and Dropbox test accounts and linked them to emulator. Generated a mock GPX route around Vancouver and imported it to simulate an activity. Activity did not appear on Strava; began troubleshooting and analyzing logs.	2.0
30th Oct 2025 (Morning)	Umesh	Investigated Strava database structure using DB Browser for SQLite. Injected mock JSON activity entry into strava-database. Encountered app crash due to missing native library (libFatmapSdk.so).	2.5
5th Nov 2025	Umesh	Uploaded prepared sample files to Dropbox to simulate user data and activity. Verified folder structure for forensic testing.	1.0
5th Nov 2025	Umesh	Installed and configured both ALEAPP and MobSF tools in the project environment. Verified successful setup and readiness for analysis.	1.5
7 th Nov 2025	Umesh	Attempted to create sample app data for Strava using emulator.	1
8 th Nov 2025	Umesh	Created sample activities for Strava account using a separate device and made it visible in the emulator.	1
9 th Nov 2025	Umesh	Pulled data/data directories of both the apps and ran ALEAPP on those folders.	1.5
9 th Nov 2025	Umesh	Analyzed permission errors for .mapbox & .fit files in Strava extraction. Reran with elevated privileges.	1

9 th Nov 2025	Umesh	Troubleshoot empty ALEAPP reports and manually inspected WAL/SQLite files for forensic artifacts.	2
11 th Nov 2025	Umesh	Attempted MobSF local server run via python and failed, then began docker setup for MobSF container	1.5
12 th Nov 2025	Umesh	Attempted dynamic analyzer configuration in MobSF. Encountered Android Runtime detection errors, emulator connection failures, and ADB remount issues. Verified that emulator was detected by MobSF, but system partition was not writable.	1
12 th Nov 2025	Umesh	Investigated MobSF dynamic analysis requirement for API level 30-based emulators and created a fresh Android Studio emulator running Android 11 (API 30) to meet the minimum supported version by MobSF.	0.5
12 th Nov 2025	Umesh	Attempted to run dynamic analysis again with API 30 emulator, but MobSF reported system partition as read-only. Reviewed MobSF documentation and emulator requirements. Documented compatibility issues for further investigation.	1

2. Description of Work Done

Oct 6 – Oct 28, 2025

During this period, the primary focus was on researching and finalizing the tools and applications required for the mobile forensic project. Initial work involved evaluating multiple forensic utilities and identifying ALEAPP (Android Logs, Events, and Protobuf Parser) and MobSF (Mobile Security Framework) as the most suitable for analyzing app data and behavior.

After finalizing the tools in consultation with the professor, I proceeded to set up the Android SDK command-line tools, including ADB, AVD Manager, SDKManager, and APKAnalyzer. A fully functional emulator was created to simulate a standard Android environment for forensic testing. (API 35 with google API)

By the end of this phase, Dropbox and Strava APKs were installed on the emulator, forming the foundation for subsequent forensic extraction and analysis. This setup ensured controlled, repeatable testing conditions for both static and dynamic data examination in later stages.

Oct 29 – Nov 8, 2025

The next phase concentrated on generating user activity data and preparing realistic test scenarios. I created a dedicated test Gmail account and separate Strava and Dropbox accounts, linking them to the emulator for data synchronization.

A mock GPX route was generated around Vancouver and imported into Strava to simulate fitness activities. However, despite successful import, the simulated activity did not appear within the app, suggesting synchronization or database-write limitations under the emulator environment.

To further investigate, I analyzed the Strava SQLite database using DB Browser for SQLite, where I attempted to manually inject mock JSON activity records. This resulted in an app crash caused by a missing native library (libFatmapSdk.so), indicating dependency mismatches with Strava's production build.

In parallel, Dropbox was configured with multiple test folders and sample files (PDFs, images, spreadsheets, and Word documents) to replicate real user data. This dataset was used to prepare the forensic extraction workflow later applied through ALEAPP and MobSF. During this time, both tools were successfully installed and configured. However, ALEAPP's dependency setup initially failed due to missing Python modules like simplekml and protobuf, which were manually resolved.

By November 8, all tools were operational, and the test environment was ready for data acquisition and analysis.

Nov 9 – Nov 11, 2025

This phase focused on data extraction, artifact analysis, and validation of forensic results. I pulled the **`/data/data/com.strava`** and **`/data/data/com.dropbox.android`** directories using ADB and executed ALEAPP parsing commands to analyze both datasets.

Initially, some extractions returned incomplete results due to permission errors (especially `.mapbox` and `.fit` files). After troubleshooting and rerunning ALEAPP with elevated privileges, I was able to successfully retrieve multiple user-level artifacts from Strava, including:

- Username
- Date of Birth
- Weight and Height details
- App configuration metadata
- Device usage records and database entries

These findings confirmed that ALEAPP can partially extract meaningful forensic artifacts even from modern versions of Strava when given complete directory access.

In contrast, the Dropbox extraction yielded limited results, as many database files were either encrypted or inaccessible under emulator permissions. I then began transitioning to MobSF to perform static and dynamic APK analysis for both applications. However, running the local MobSF server via Python failed due to updated version restrictions, leading me to prepare a Docker-based setup for the next testing cycle.

Nov 12th, 2025

On November 12th, I focused entirely on configuring and testing the Dynamic Analyzer in MobSF and preparing an environment compatible with its requirements. I began by attempting to run the dynamic analyzer within the MobSF Docker container. Although the container launched successfully, MobSF was unable to detect a valid Android runtime environment. The system displayed repeated errors such as *“Android Runtime Not Found”*, *“Cannot Connect to host.docker.internal:5555”*, and *ADB remount failures*, indicating that communication between MobSF and the emulator was not fully established.

To troubleshoot these issues, I verified emulator connectivity using ADB from inside the container and confirmed that MobSF could at least detect the emulator instance. However, MobSF flagged that the emulator’s /system partition was read-only, which prevented the required instrumentation steps (runtime patching, agent installation, API hooking). This made the emulator unsuitable for dynamic analysis.

Upon reviewing MobSF’s documentation, I learned that the tool supports Android 11 (API 30) as the minimum version for dynamic analysis. I therefore created a new emulator running API 30, ensuring compatibility with MobSF’s expected Android runtime version. Once the emulator was set up, I attempted dynamic analysis again, but MobSF continued to report that the system partition was not writable and therefore could not proceed.

I documented these compatibility issues, the errors encountered (runtime detection, connection failures, remount errors), and the tests performed. This completes the investigation phase for MobSF dynamic analysis and establishes the need to continue with static analysis while researching a workaround for emulator system write access.

3. AI Use Section

Table of AI Tools and Specific Use:

AI Tool Name	Version, Account Type	Specific feature for which AI tool was used
ChatGPT	GPT-5 (Plus)	Helped generate GPX routes, create sample Dropbox/Strava data; assisted in troubleshooting ADB, ALEAPP, and MobSF setup; helped interpret Android logcat errors; generated worklog entries and report wording.

Although AI tools were used to guide troubleshooting, generate documentation, and simplify explanations, all final decisions, implementations, and forensic workflows were performed manually by me. This includes:

1. Manual Environment Setup & Debugging
2. Hands-On Forensic Procedures
3. Independent Analysis & Interpretation
4. Report Writing & Academic Contributions

AI provided suggestions and explanations, but I performed all technical actions, verifications, testing cycles, file manipulations, and forensic validation myself, ensuring the analysis remained academically original and hands-on.

4. Appendix

“I need to create a mock gps route around Vancouver in .gpx format”

“Error: adb connect host.docker.internal:5555 explain the cause and fix for MobSF dynamic analysis.”

“Explain how to remount /system as writable in an Android emulator.”

“Why does MobSF say ‘VM’s /system is not writable’ and how can I fix this?”

“How do I run a static analysis on an APK using MobSF if dynamic analysis is not working?”

“ALEAPP produced empty reports for Strava and Dropbox. What files do I need to include for successful artifact extraction?”